

Hlidskjalf

SCADA System Proof Of Concept

Magne Mardal Moe

Mail: mamoe4908@usn.no /
magne@mardal.moe

May 18. 2026

Course IIA2017
Industrial Information Technology

Faculty of Technology, Natural Sciences and Maritime Sciences
TNM - Campus Porsgrunn

I. ABSTRACT

This paper presents the development of a Proof-of-concept SCADA system. With standard SCADA components such as Controll System, Data logger and Alarm/monitoring system (HMI). The project could be divided into five phases, problem analysis, project design, project development, security hardening and system analysis. To solve the problem in this project developed systems were integrated, such as LabView, MS-SQL database server and .Net10. The project is seen as a success, but there are further work to make it viable as a fully deployed system.

CONTENTS

I	Abstract	1
II	Introduction	2
III	Materials and Methods	2
III-A	Materials	2
III-B	Methods	2
III-B1	Research design . .	2
III-B2	Use of AI	2
IV	Database	3
IV-A	Design	3
IV-B	Database server	3
IV-C	Adjustments	3
IV-D	Views	3
IV-E	Stored Procedures	3
V	Bifrost: The Control System	4
V-A	Design	4
V-B	PI(D)-Controller	4
V-C	Low-Pass filter	4
V-D	Simulation	5
V-E	Real life data	5
VI	Muninn: The Datalogger	5
VI-A	Design	5
VI-B	Front panel	5
VI-C	Timestamp	5
VII	Huginn: The Alarm & Monitoring Application	6
VII-A	Design	6
VII-B	Fuctions	6
VII-B1	Authentication . . .	6
VII-B2	Add monitors . . .	6
VII-B3	Config data	6
VII-B4	Alarms	6
VIII	Cyber security	6
VIII-A	OPC UA Server	6
VIII-B	Alarm & Monitoring System .	7
IX	Discussions	7
X	Conclusions	7
X-A	Further Work	7

XI References

8

LIST OF FIGURES

1	System Diagram	2
2	Initial database design	3
3	finished Database design	3
4	Bifrost Front Panel	4
5	PI Controller	5
6	Muninn Front panel	6

LIST OF TABLES

I	Hardware	2
II	Software	2

TABLE I: Hardware

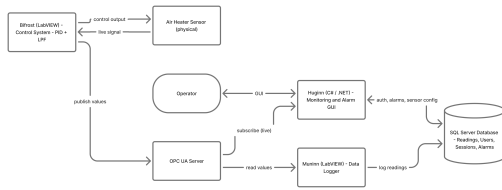
Type	Model
Computer	Lenovo P14s Gen5
Computer	Desktop-computer
DAQ	NI USB-6008
Process Plant	Airheater
Proxmox Server	Minisforum MS-01

TABLE II: Software

Software	Version
Windows 11	25H2
Labview	26Q1
Visual Studio 2026	18.2.1
OPC UA server	1.2.0
OPC UA client	1.4.0
Docker	28.4.0
MSSQL container	24.0.4

II. INTRODUCTION

Supervisory Control and Data Acquisition (SCADA) System has become a staple in modern industrial processes, providing control, logging and alarm functionality. This report will introduce the SCADA System Hlidskjalf which contains three main programs, a control system, a Data logger system and an Alarm and Monitoring system, where all systems contains a Human-Machine Interface (HMI). Project files can be viewed and downloaded at Github.

**Fig. 1: System Diagram****III. MATERIALS AND METHODS****A. Materials**

Materials and software used in this project is listed in the tables below.

B. Methods**1) Research design**

This project was made in five phases.

- Phase 1: Problem analysis
- Phase 2: Project design
- Phase 3: Project development
- Phase 4: Security Hardening
- Phase 5: System analysis

These phases were chosen to get a structured project for best results. At the start problem analysis gave an overview of the steps necessary to get a successful project. The project design phase gave an overview of the steps needed to develop and what those steps should look like. The project development phase was the longest phase where the programs in this project was developed and slightly tested. To get a system with minimal security holes, the security hardening phase gave space to edit the programs. The last phase, system analysis, gave room for testing and deployment on hardware to make sure the programs ran successful on different systems.

2) Use of AI

The AI model OPUS 4.7 has been used for problem solving discussions, and did not contribute to text generation in this report. Some code were generated to help understand some functionality, but has not been directly pasted into any programs in this assignment. The model was used to write the README.md file on the github repository.

IV. DATABASE

A. Design

The database was designed in the web application dbdesigner. The application provides a gui to design the database without using Structured Query language (SQL). This makes it easier to design and visualize the database before deploying it to a database server. The initial database design can be viewed in Figure 2. Without a payed subscription to dbdesigner it only supports 2 database models with a maximum of 10 tables pr model. The app includes a function where the designed database can be exported as SQL for easy deployment.

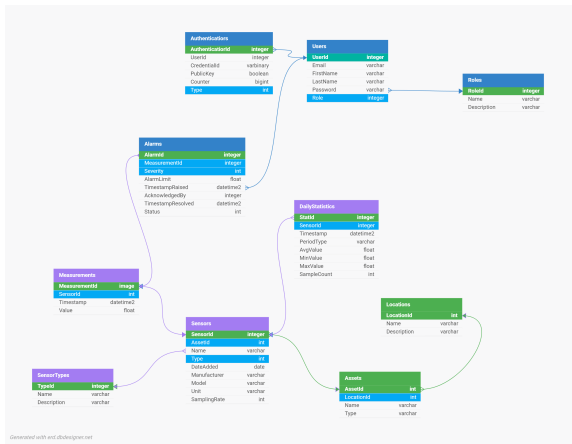


Fig. 2: Initial database design

B. Database server

The database design was exported from dbdesigner, and was deployed to a Microsoft SQL server running in a Docker container on a virtual machine on a Minisforum MS-01 Microcomputer running proxmox (operating system). This method was preferred since the development of the system was made on two different computers and by using a server a common database could be used even in development.

C. Adjustments

Since the design for the database only supported 10 tables there was made some adjustments to make the

database work better with the applications. There was added some more tables, some changes in data types and som changes in the columns of the tables. While most of the tables is used in the programs some is not yet implemented, like authenticators which is planned as a two factor authentication or passkey storage for the users.

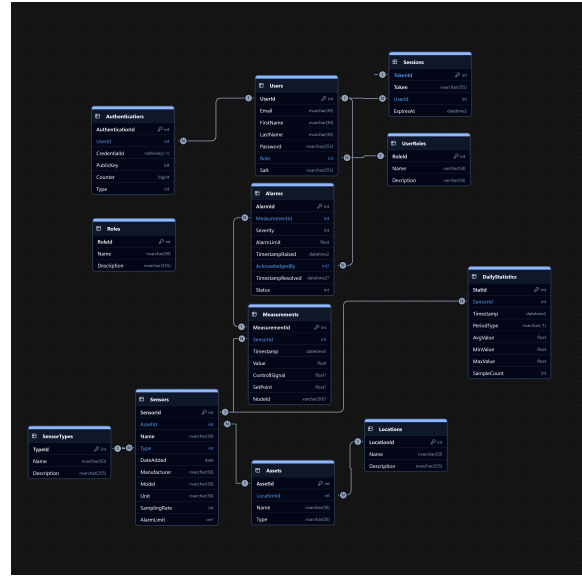


Fig. 3: finished Database design

D. Views

In the database there was made four database views, some to make it easier to read rows from the tables for humans, for example a view to view the measurements with data from the sensor which the measurement was made. Some were made to get data for the programs for example a view for finding measurement ID from sensor name, value and timestamp.

E. Stored Procedures

There was made four stored procedures in the database. These were made to create sensors, measurements and alarms, and one to update an alarm so it is registered as seen. This makes it easier to add statements to the code in the programs described later

in this report. The stored procedures also makes it so the programs do not need to make ass many calls to the database to execute requests, to for example add a sensor with a location. Where it is only necessary to know what asset name the sensor should be stored ass, and not the assetid which is stored in the sensor table. For the sensors, it will also make a name for the sensors automatically so it follows a standard naming practise. The stored procedures can be viewed in ??.

V. BIFROST: THE CONTROL SYSTEM

The application Bifrost is the control system Application of the SCADA system. The purpose of this application is to read value from a sensor, and save the data to a OPC UA server. This application is also able to controll the sensor using a PI(d).

A. Design

Bifrost is made in LabView, with a easy to use graphical user interface (GUI) to both control the sensors and to view a graph of the values measured. This labview project includes a main VI and multiple SubVIs to simplify the back panel of the main file, and to make it possible to use the same SubVis for multiple projects.

When the program is runned, it wil start with an initial state which will clear the graph and set the logging to false. The program wil then ask for credentials to the OPC UA server before connecting to it. After the application is finished with inital setup it wil enter a *wait* state where it check for user inputs. If the is started as running, it will go to the *getData* state and either simulate the data or read/write to the sensor/actuator before saving the data in a OPC UA server. As long as the program is in the running state it will continue this until it is set in the idle state. The interval of getting and saving data is configured by the sampling time. The front panel of the controll system can be seen in Figure 4.

B. PI(D)-Controller

A Proportional Integral Derivative Controller (PID Controller) is used for feedback control in industrial automation. Proportional term responds to error by generating a action based on the error, the Integral

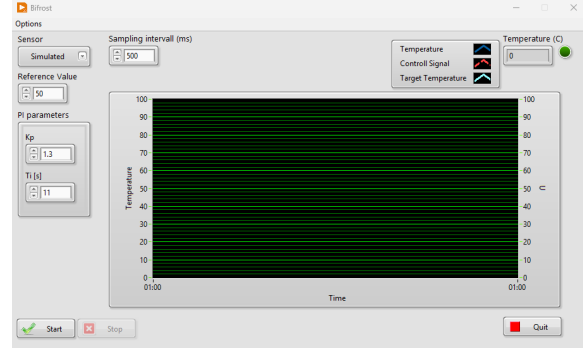


Fig. 4: Bifrost Front Panel

part takes the past error over time and generate a signal to eliminate error. This means that small errors are corrected over time. The Derivative part is used to predict future error by measuring the change in error [1]. In this project the controller is not designed to integrate and will therefor be called the PI controller further on.

The PI Controller, block diagram shown in Figure 5, is made with a *Formula Node*. This makes it easy to create mathematical formulas to use with given data. The two formulas in the Formula node is derived from the formula for a PID controller ass seen under.

$$u(t) = K_p e + \frac{K_p}{T_i} \int_0^t e d\tau + K_p T_d \dot{e} \quad (1)$$

In equation (1) the $K_p e$ is the proportional part, $\frac{K_p}{T_i} \int_0^t e d\tau$ is the integral part and $K_p T_d \dot{e}$ is the derivative part. In this controller, since it does not contain a derivative part, the functions can be implemented in LabView with the following equations (2) and (3).[3]

The

$$e_k = r_k - y_k \quad (2)$$

$$u_k = u_{k-1} + K_p(e_k - e_{k-1}) + \frac{K_p}{T_i} T_s e_k \quad (3)$$

C. Low-Pass filter

To eliminate noise from physical devices, it was implemented a Low-Pass Filter (LPF). A first order

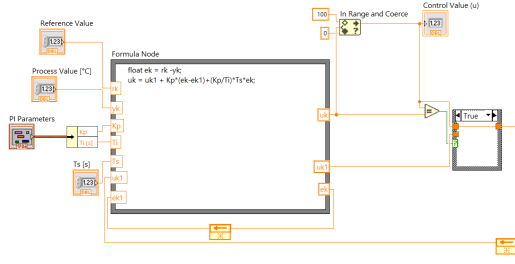


Fig. 5: PI Controller

IIR low-pass filter was applied between the sensor output and the PI controller. This filter also uses a formula node to calculate the output value.[2]

$$y_k = \alpha x_k + (1 - \alpha)y_{k-1} \quad (4)$$

$$\alpha = \frac{\Delta t}{RC + \Delta t} \quad (5)$$

The output of the low pass filter is calculated with equation (4). The smoothing factor α is calculated with equation (5). $\Delta t = \frac{1}{f_s}$ and $RC = \frac{1}{2\pi f_c}$ where f_c is the cut-off frequency and f_s is the sampling frequency.

$$RC = \frac{1}{2\pi f_c} \quad (6)$$

D. Simulation

Because of access to physical equipment the application was developed while using simulated data. The simulation was made with a mathematical equation which should match how a AirHeater would function with control signal and output. The simulation program was made with a formula node with equation (7) where $u_{delayed}$ is the output value from the PI Controller. In this simulation the ambient temperature is at the fixed point 20°C. This was done for simplicity, but in a real world case, the air heater would heat the ambient temperature as well [8].

$$T_{next} = T + \Delta t \frac{((t_{amb} - T) + K_h * u_{delayed})}{t_c} \quad (7)$$

E. Real life data

An early developed version of the programs was tested using a hardware airheater to test if the programs works with real world equipment, and not only simulated data. This was done at the electronics lab at USN campus Porsgrunn. This test was successful and also gave some good inputs for further development of the application.

VI. MUNINN: THE DATALOGGER

A. Design

The Datalogger was also made in LabView as a case machine, where it has an idle state and a running state. When the program is launched the init state resets the data to initial values and checks a config file to find the setup last used. It will then open a page where login information for the OPC UA server is inputted. Then it will connect to the OPC UA server and the Database given in the config file. The settings for the OPC server and the database can be edited in the runtime menu and is saved to a config file.

B. Front panel

When the program is set in running state it will read data from the OPC UA server based on the Node Ids given, and save them in a database using stored procedure based on the sensor name which is set by the user, this parameters is read from the database to ensure the sensors that are being written to actually exists. The data logger also gives the opportunity to register (create) new sensors in the database. And also supports logging multiple sensors at the same time, but it only supports one sampling time for the system at a time.

The program will show all the current sensors that are configured to log in a table. The table displays the sensor name from the database, the node id configured to gather data from and the value and timestamp from the OPC UA server.

C. Timestamp

The computers used in development of this application had different time format, it was discovered that

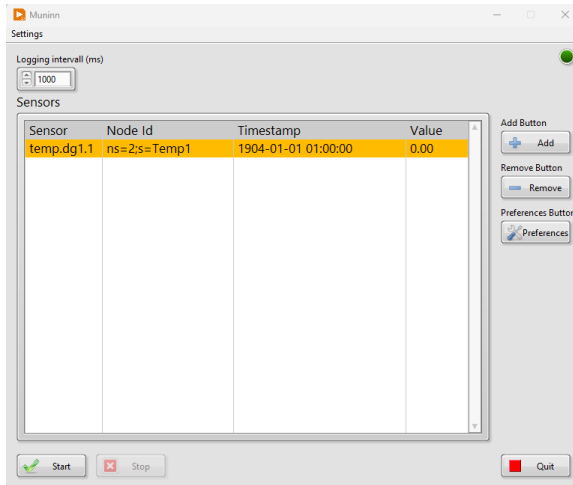


Fig. 6: Muninn Front panel

LabView gathers the system clock of the host system. While using OPC this was not a problem, but when it should write to the database it became clear that the format of the timestamp had to be corrected and set as *MM/DD/YYYY* to make sure that the program was not dependent on the date format of the host system.

VII. HUGINN: THE ALARM & MONITORING APPLICATION

A. Design

The alarm & Monitoring application was developed as a Windows Forms Application using C# in visual studio with the .Net10 framework from Microsoft. It was preferable to read raw data from the OPC UA server rather than use data from the databasem, this ensures that any alarms, and the values that is being monitored are based on the newest available data. It will also make it possible for the monitoring system to function even if the data logger is offline or fails. Alarms should still be saved in the database, and to make sure the alarms are saved correctly, there was som logic that had to be implemented. It uses some database views as well as stored procedures to find a measurement that matches the sensor, value and timestamp of the data read from the OPC server. Then it uses that value to link an alarm to a measurement.

B. Fuctions

1) Authentication

Huginn uses credentials stored in the database to authenticate users. The application will make a session for the user and store session data in the database, this session has an seven day expiration. While the session is active the user can close and reopen the program without needing to log in again. The users have a role that gives access, and there is also an admin role which is intended to be able to create new users. As of this report this function is not yet implemented.

2) Add monitors

The application can simultaneously monitor nine sensors at a time. The sensors selected will then read the last measurement saved to get the Node Id, but this value can be overwritten. These sensors are then displayed as a gauge with a value indicator. This makes it easier for an operator to take a glance and get meaningful data.

3) Config data

The program uses a custom made config manager that reads data from a config file to get data on the OPC server and database that should be used, as well as the sensors that is being monitored.

4) Alarms

The system generates alarms based on the stored alarm limit value, if the value read from the OPC server exides that value, it will create an alarm, display it in a table and store it in the database. The alarm will then be visible with an acknowledge button which when pressed will update the status of the alarm and store the user authenticated users within the alarm. It will also record the date and time the alarm was acknowledged to make sure the data is tracked. The *Ack* button will then be replaced by a resolve button which when pressed updates the alarm status and hides the alarm from the table. It is also a function that prevents a new alarm to be created if there already is an either active or acknowledged alarm present.

VIII. CYBER SECURITY

A. OPC UA Server

The OPC UA server is password protected and needs users to authenticate to get access. OPC server

and clients supports any certificate, which also should be improved to make sure certificates is valid and trusted.

B. Alarm & Monitoring System

The Alarm & Monitoring application is password protected with session/token data stored. The credentials to the OPC UA server and the database is stored in an user specific folder in *Appdata*. This makes sure that the system is protected, but the credentials for the servers is still written in plain text, which should be updated with a form of encryption.

When authenticating the username and password it will use a password hash to make sure the password is not sent over the network in plain text.

IX. DISCUSSIONS

The Hlidskjalf SCADA system demonstrates a functional proof of concept using a combination of LabView for control and data logging, MS-SQL for persistent data storage and .Net10 windows Forms application for alarm and monitoring. The architecture is modular where every application should be able to run as independent applications communicating over a network between OPC UA server, database server, and computer programs.

The choice of OPC UA as the communication standard was a key design. It gives a standardized interface between the control system and the other applications. However the missing security aspect considering certificate validation is critical to prevent man-in-the-middle attacks.

The database design went through several iterations during development. The initial design was made in dbdesigner, which limits a free user to ten tables per database model. This forced some compromises that had to be corrected later in development. The use of stored procedures and database views proved valuable, both for reducing the database calls from the applications and making the logic easier to develop and understand.

The security aspect of the system is currently based on basic mechanisms expected from a system; password authentication on OPC server, database server and the alarm and monitoring application. The

use of hashed passwords for authenticating users is necessary to keep user data secure. However there are still some system passwords stored in plain text inside the applications and on the computer running the applications, which is critical to improve to ensure security. Two-factor authentication and passkey support were planned in the database design but not implemented in the current version of the system.

X. CONCLUSIONS

The system is build with three integrated programs: Bifrost for control, Muninn for data logging and Huginn for alarm & monitoring. The system meets the core functions of a SCADA system, giving supervisory control, persistent data storage and real-time monitoring with alarm functionality. The system is designed using simulated data and tested with a physical air heater to confirm the system works in practice.

The five phase research design provided a structured development process that kept the project on track. The use of OPC UA makes the program more flexible with different sensors and systems, and by using the real-time data functionality gives opportunity for fast action and error handling.

While the system functions as intended it should still be considered as a proof-of-concept rather than a deployment-ready product. There are several limitations, including within security, scalability and user management, which should be improved before the system could be implemented in an industrial environment.

A. Further Work

Several areas have been identified for further development:

- Certificate validation for the OPC UA server.
- Encryption of credentials to the OPC Server and database in Huginn.
- Two-factor authentication and passkey functionality for the Alarm & monitoring application to ensure security in the application.
- Error handling in all systems.
- Admin functionality in Huginn to make it possible to create and manage users, which is currently unimplemented.

- Testing on physical devices to make sure the system works over time and gives stability on physical devices.
- Improve the UI to be more intuitive and better looking.

XI. REFERENCES

- [1]GeeksforGeeks, “Proportional Integral Derivative Controller in Control System,” GeeksforGeeks, Oct. 24, 2023. <https://www.geeksforgeeks.org/electronics-engineering/proportional-integral-derivative-controller-in-control-system/> (accessed May 12, 2026).
- [2]C. Wanderer, “Low Pass Filter Explained,” Medium, Aug. 22, 2025. <https://medium.com/@cosmicwanderer/low-pass-filter-explained-18f673d4d844> (accessed May 14, 2026).
- [3]The Technical Guy, “Discrete Control Systems in LabVIEW,” YouTube, Mar. 31, 2023. <https://www.youtube.com/watch?v=rqKdInEqXdo> (accessed May 10, 2026).
- [4]The Technical Guy, “LabVIEW OPC UA,” YouTube, Feb. 02, 2021. <https://www.youtube.com/watch?v=AoneC2cSzto> (accessed May 09, 2026).
- [5]The Technical Guy, “OPC UA with Visual Studio and C#,” YouTube, Feb. 18, 2022. <https://www.youtube.com/watch?v=KCW23eq4auw> (accessed May 10, 2026).
- [6]The Technical Guy, “LabVIEW SQL Toolkit,” YouTube, Apr. 15, 2024. <https://www.youtube.com/watch?v=OMFWWXdeY> (accessed May 08, 2026).
- [7]The Technical Guy, “SQL Server with C# Windows Forms App,” YouTube, Jun. 22, 2021. <https://www.youtube.com/watch?v=rXugzELsQl0> (accessed May 12, 2026).
- [8]F. A. Haugen, “Air Heater,” Techteach.no, 2024. https://techteach.no/lab/air_heater/ (accessed May 05, 2026).